```
%cd "C:/Users/11584/Desktop/OMGroup/ESE023"
import os
# import pandas
import pandas as pd
# import numpy
import numpy as np
# import matplotlib
from matplotlib import pyplot as plt
# make plots appear and be stored within the notebook
%matplotlib inline
os.getcwd()
```

1.1 [5 points] Compute the total number of deaths caused by earthquakes since 2150 B.C. in each country, and then print the top 20 countries along with the total number of deaths.

```
##PS2_1_1

#met = pd.read_xlsx("earthquakes-2026-10-14_20-28-44_+0800.tsv")
Sig_Eqs = pd.read_csv("earthquakes-2026-10-14_20-28-44_+0800.tsv", sep = "\t")
Sig_Eqs1 = Sig_Eqs[['Deaths','Country']]
#print(Sig_Eqs1)
#Sig_Eqs2 = Sig_Eqs1.groupby(by = ['Country'], as_index=False).sum(min_count =
1).sort_values(['Deaths'], ascending = False)
#if not defined, every index will be sum
Sig_Eqs2 = Sig_Eqs1.groupby(by = ['Country'], as_index = False)['Deaths'].sum(min_count
= 1).sort_values(['Deaths'], ascending = False)
#Sig_Eqs2.head(20)
Top_20 = Sig_Eqs2[0:20]
print(Top_20)
```

```
     Country     Deaths
28     CHINA  2139209.0
146   TURKEY  1199720.0
```

68 IRAN 1014453.0
72 ITALY 492236.0
136 SYRIA 419826.0
61 HAITI 323484.0
10 AZERBAIJAN 319251.0
74 JAPAN 242516.0
6 ARMENIA 177890.0
106 PAKISTAN 145085.0
69 IRAQ 136200.0
41 ECUADOR 135496.0
147 TURKMENISTAN 117412.0
110 PERU 102309.0
71 ISRAEL 93388.0
113 PORTUGAL 83572.0
55 GREECE 81162.0
27 CHILE 64277.0
65 INDIA 63507.0
137 TAIWAN 57154.0

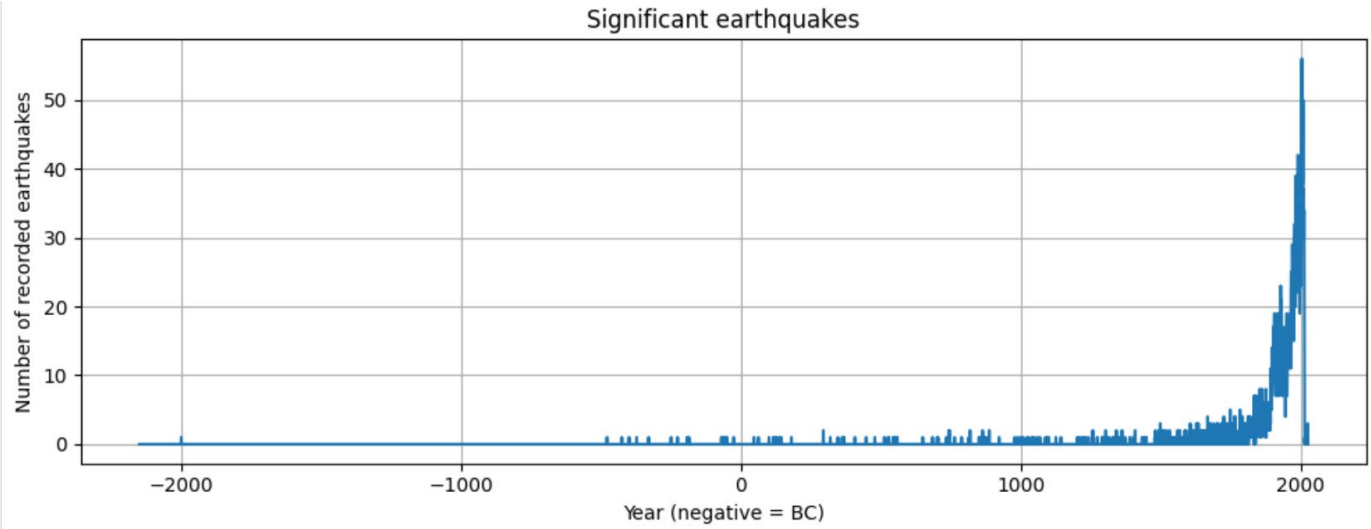
1.2 [10 points] Compute the total number of earthquakes with magnitude larger than 3.0 (use column Ms as the magnitude) worldwide each year, and then plot the time series. Do you observe any trend? Explain why or why not?

```
##PS2_1_2

Sig_Eqs1 = Sig_Eqs[['Ms','Year']]
large_eq = Sig_Eqs1.loc[ (Sig_Eqs1['Ms'] > 3) ]
eq_count = large_eq.groupby("Year").size()
#fill a zero into the year without a Sig_eq
last_year = int(Sig_Eqs["Year"].max())
years = range(-2150, last_year + 1)
eq_count = eq_count.reindex(years, fill_value=0)
```

```
plt.figure(figsize=(10, 4))
plt.plot(eq_count.index, eq_count.values)
plt.xlabel("Year (negative = BC)")
plt.ylabel("Number of recorded earthquakes")
plt.title("Significant earthquakes")
plt.grid(True)
plt.tight_layout()
plt.show()
```

```
#trend: 强地震记录在 1500 年开始逐年递增，
#到 2000 年达到最高峰，从 2000 年以后断崖式下跌.
#接下来分析原因：1500 年以后，地震记录可能才开始完善，
#并且记录在后来才能有效保存，越接近古代，能得到保存的记录就越少
```




```
##PS2_1_2
```

```
Sig_Eqs1 = Sig_Eqs[['Ms', 'Year']]
```

```
#筛选 2000 年及以后的全部地震记录
total_eq = Sig_Eqs1.loc[Sig_Eqs1['Year'] >= 2000]
```

```
#在这些记录中筛选 Ms>3 的地震
```

```
large_eq = total_eq.loc[total_eq['Ms'] > 3]

#分别统计每年的次数
total_count = total_eq.groupby('Year').size()
large_count = large_eq.groupby('Year').size()

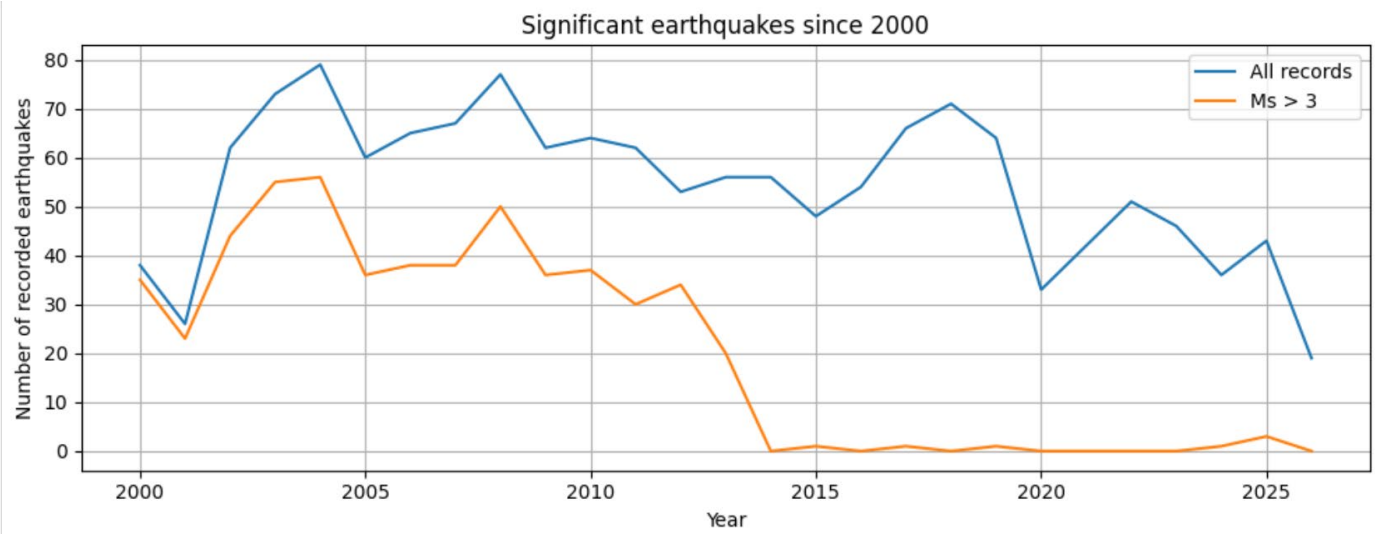
#补齐年份，没有符合条件记录的年份填 0
last_year = int(Sig_Eqs['Year'].max() )
years = range(2000, last_year + 1)

total_count = total_count.reindex(years, fill_value = 0 )
large_count = large_count.reindex(years, fill_value = 0 )

#连续调用两次 plot，将两条线画在同一张图上
plt.figure(figsize=(10, 4))
plt.plot(total_count.index, total_count.values, label = 'All records')
plt.plot(large_count.index, large_count.values, label = 'Ms > 3')

plt.xlabel('Year')
plt.ylabel( 'Number of recorded earthquakes')
plt.title('Significant earthquakes since 2000' )
plt.legend()
plt.grid( True )
plt.tight_layout()
plt.show()

#数据分析：自 2014 年开始，在总地震记录数目无显著下跌的情况下，
#强地震次数趋近于 0.可能是强地震统计口径发生了变化
```



1.3 [10 points] Write a function CountEq_LargestEq that returns (a) the total number of earthquakes since 2150 B.C. in a given country AND (b) date and location of the largest earthquake ever happened in this country. Apply CountEq_LargestEq to every country in the file, report your results in a descending order.

```
##PS2_1_3 definition
def CountEq_LargestEq(country):
    country_eq = Sig_Eqs[ Sig_Eqs[ "Country"]  == country]
    total = len(country_eq)

    #delete countries without records
    known_eq = country_eq.dropna(subset=["Mag"])
    if len(known_eq) == 0:
        return total, np.nan, np.nan, np.nan, np.nan, "Unknown"
    if country == "TAIWAN":
        print("TAIWAN is a region of China with independent records.")

    largest_index = known_eq["Mag"].idxmax()
    largest = known_eq.loc[largest_index]
    magnitude = largest[ "Mag" ]
    year = largest["Year"]
    month = largest["Mo"]
```

```
    day = largest[ "Dy"]
    location = largest["Location Name"]
    return total, magnitude, year, month, day, location

##PS2_1_3
# get the country list
countries = Sig_Eqs["Country"].dropna().unique()
results = []
# loop for every country
for country in countries:
    total, magnitude, year, month, day, location = CountEq_LargestEq(country)
    results.append([country, total, magnitude, year, month, day, location])

columns = ["Country", "Total_frequency", "Max_Mag", "Year", "Month", "Day", "Location"]
country_results = pd.DataFrame(results, columns = columns)
# transfer date into integer
country_results["Year"] = country_results["Year"].astype("Int64")
country_results["Month"] = country_results["Month"].astype("Int64")
country_results["Day"] = country_results["Day"].astype("Int64")

country_results = country_results.sort_values("Total_frequency", ascending = False)
country_results = country_results.reset_index(drop = True)


print(country_results.to_string)
```

```
TAIWAN is a region of China with independent records.
<bound method DataFrame.to_string of             Country  Total_frequency  Max_Mag  Year  Mont
h  Day  \
0             CHINA              626      8.5  1668      7   25
1             JAPAN              429      9.1  2011      3   11
2         INDONESIA              427      9.1  2004     12   26
3              IRAN              385      7.9   856     12   22
4            TURKEY              359      7.8  1939     12   26
..              ...              ...      ...   ...    ...  ...
157         BURUNDI                1      4.7  2004      2   24
158  CZECH REPUBLIC                1      4.1  2008     11   22
159      MADAGASCAR                1      5.5  2017      1   11
160          ZAMBIA                1      5.9  2017      2   24
```

```
161         COMOROS                1      5.9  2018      5   15

                                      Location
0                       CHINA:  SHANDONG PROVINCE
1                                 JAPAN:  HONSHU
2    INDONESIA:  SUMATRA:  ACEH:  OFF WEST COAST
3                           IRAN:  DAMGHAN, QUMIS
4                               TURKEY:  ERZINCAN
..                                             ...
157                            BURUNDI:  RUYAGA
158                     CZECH REPUBLIC:  KARVINA
159                        MADAGASCAR: ANTSIRABE
160                              ZAMBIA:  KAPUTA
161                             COMOROS:  MAYOTTE

[162 rows x 7 columns]>
```

2. Air temperature in Shenzhen 1998-2022

In this problem set, we will examine how air temperature changes in Shenzhen from 1998 to 2022 using the hourly weather data measured at the Baoan International Airport. The data set is from NOAA Integrated Surface Dataset. Download the zipped file Baoan_Weather_1998_2022.zip, unzip it, and move the Baoan_Weather_1998_2022.csv file to your working directory.

Read page 10-11 (POS 88-92 and POS 93-93) of the comprehensive user guide for the detailed format of the air temperature data (use column TMP). Explain how you filter the data in your report.

[10 points] Plot monthly averaged air temperature against the observation time. Is there a trend in monthly averaged air temperature from 1998 to 2022?

```
##PS2_2 data tidy
weather = pd.read_csv("data/Baoan_Weather_1998_2022.csv", low_memory = False)
weather1 = weather[['DATE','TMP']]
temperatures = []
quality_codes = []
# +0618,1 prior 5 code refer to TMP. the last code refer to quality code
for value in weather["TMP"]:
    temperature = int(value[:5])
    quality = value[6]
    temperatures.append(temperature)
    quality_codes.append(quality)

weather["Temperature"] = temperatures
```

```
weather["Quality"] = quality_codes

# transfer codes to temperature
weather = weather.dropna(subset=["Temperature"])
weather = weather[weather["Temperature"] != 9999]
weather = weather[weather["Quality"].isin(["0", "1", "4", "5", "9"])]
# 1,5: good quality; 0,4: limit check data; 9: pass if it's present
weather["Temperature"] = weather["Temperature"] / 10
weather = weather[weather["Temperature"] >= -93.2]
weather = weather[weather["Temperature"] <= 61.8]
# range definition from POS 88-92

same_time = weather.groupby("DATE")["Temperature"].mean()
same_time = same_time.reset_index()
# more than one record in the same time

# mean by hours
hours = []
for date in same_time["DATE"]:
    hours.append(date[:13])
# 1998-01-01T00
# 01234567890123. a 12-digits code
same_time["Hour"] = hours
hourly_temp = same_time.groupby("Hour")["Temperature"].mean()
hourly_temp = hourly_temp.reset_index()

# mean by months
months = []
for hour in hourly_temp["Hour"]:
    months.append(hour[:7])
# 1998-01-01T00
# 01234567890123. a 12-digits code
hourly_temp["Month"] = months
```

```
monthly_temp = hourly_temp.groupby("Month")["Temperature"].mean()

all_months = []
month_numbers = ["01", "02", "03", "04", "05", "06", "07", "08", "09", "10", "11", "12"]
for year in range(1998, 2023):
    for month in month_numbers:
        all_months.append(str(year) + "-" + month)
monthly_temp = monthly_temp.reindex(all_months)

print("有效记录数：", len(weather))
print("最后一个有效观测时刻：", weather["DATE"].max())
print(monthly_temp.head(12))

##PS2_2 ploting

plt.figure(figsize=(10, 4))
plt.plot(monthly_temp.index, monthly_temp.values)

plt.xlabel("Time")
plt.ylabel("Monthly mean temperature (°C)")
plt.title("Baoan Airport monthly temperature, 1998-2022")
plt.show()

# trend analysis
# bar plotting with coldest and hottest month of every year
# the coldest month is mainly on January,
# while the hottest month is on July or October
year_records = []
hot_count = [0] * 12
cold_count = [0] * 12

for year in range(1998, 2023):
    year_temp = monthly_temp[str(year) + "-01":str(year) + "-12"]
```

```
    year_temp = year_temp.dropna()
    highest = year_temp.max()
    lowest = year_temp.min()
    hot_months = []
    cold_months = []

    for date in year_temp.index:
        month = int(date[5:7])

        if year_temp[date] == highest:
            hot_months.append(month)
            if len(year_temp) == 12:
                hot_count[month - 1] = hot_count[month - 1] + 1

        if year_temp[date] == lowest:
            cold_months.append(month)
            if len(year_temp) == 12:
                cold_count[month - 1] = cold_count[month - 1] + 1

    year_records.append([year, hot_months, cold_months, highest, lowest,
                         len(year_temp), len(year_temp) == 12])

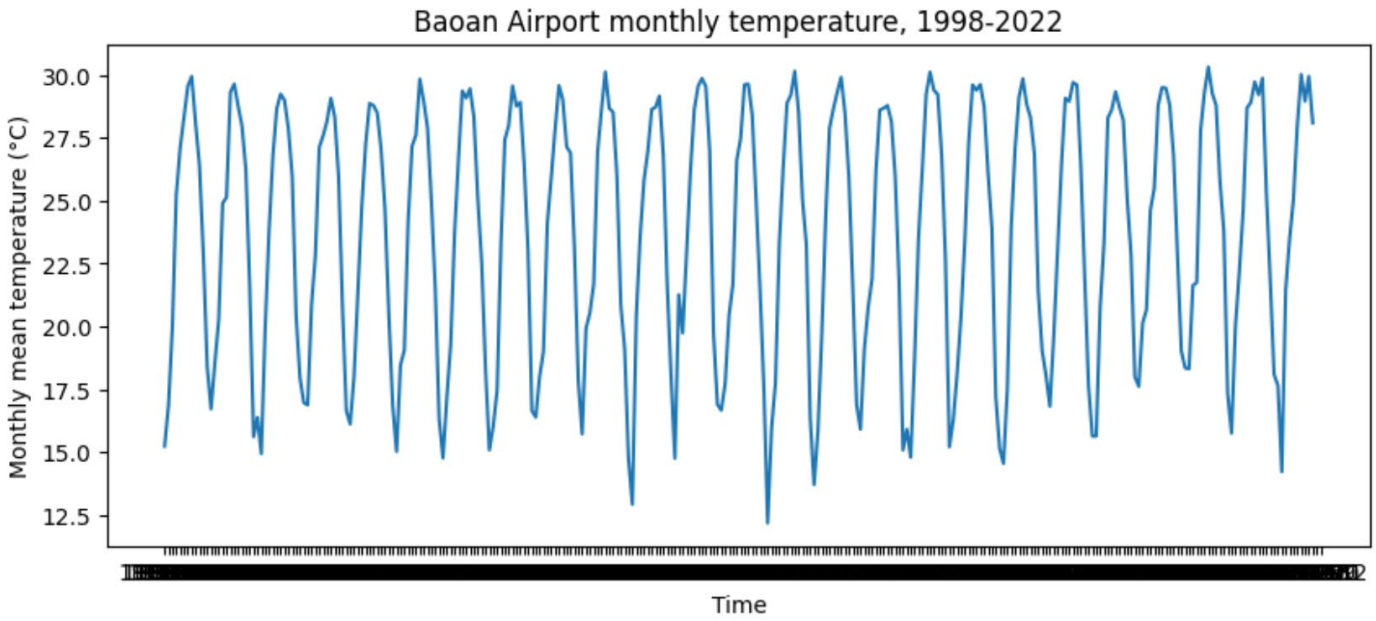
columns = ["Year", "Warmest_months", "Coldest_months", "Highest_monthly_mean_C",
           "Lowest_monthly_mean_C", "Valid_months", "Included_in_frequency"]
year_months = pd.DataFrame(year_records, columns=columns)
print(year_months.to_string(index=False))

frequency = pd.DataFrame()
frequency["Month"] = range(1, 13)
frequency["Warmest_count"] = hot_count
frequency["Coldest_count"] = cold_count
print(frequency.to_string(index=False))
hot_positions = []
```

```
cold_positions = []
for month in range(1, 13):
    hot_positions.append(month - 0.2)
    cold_positions.append(month + 0.2)

plt.figure(figsize=(10, 5))
plt.bar(hot_positions, hot_count, width=0.4, color="red", label="Warmest month")
plt.bar(cold_positions, cold_count, width=0.4, color="blue", label="Coldest month")
plt.xticks(range(1, 13))
plt.yticks(range(0, max(hot_count + cold_count) + 1))
plt.xlabel("Month")
plt.ylabel("Frequency (number of years)")
plt.title("Baoan: warmest and coldest months, 1998-2021")
plt.legend()
plt.grid(axis="y", alpha=0.3)
plt.tight_layout()
plt.show()
```

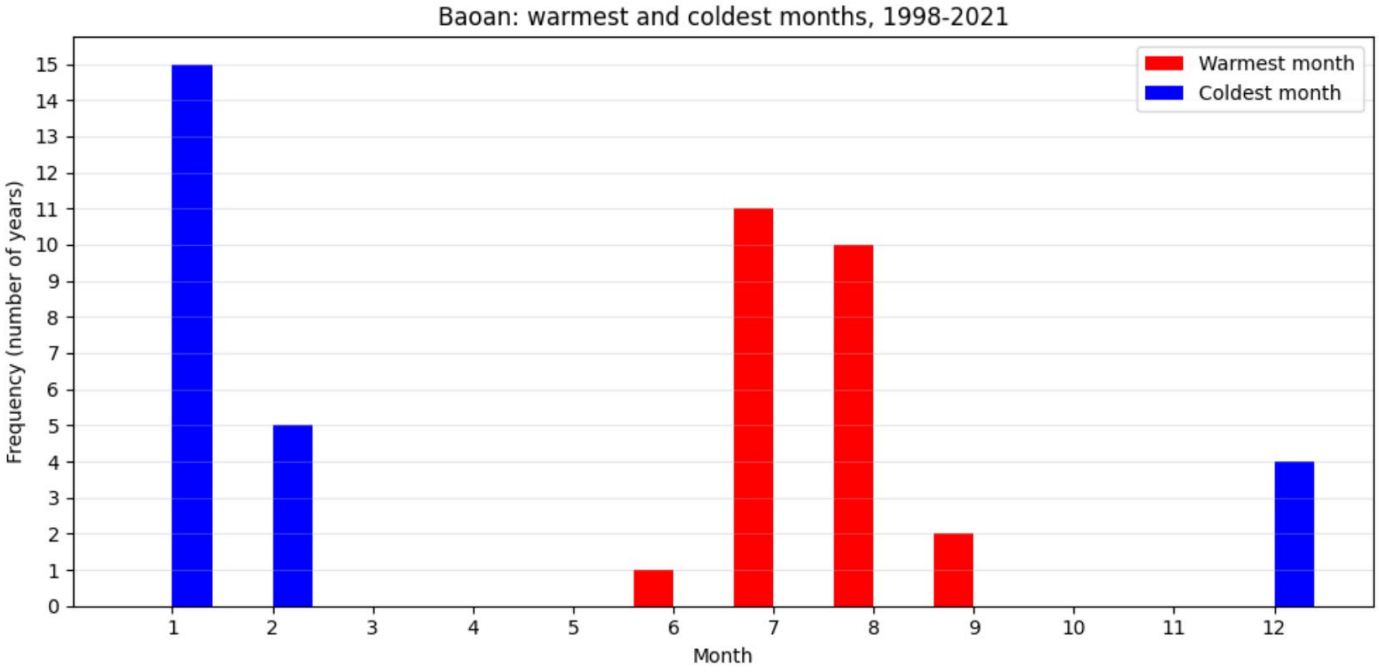
```
有效记录数： 234864
最后一个有效观测时刻： 2022-10-10T23:00:00
Month
1998-01    15.233447
1998-02    16.875304
1998-03    19.971246
1998-04    25.228365
1998-05    27.098454
1998-06    28.313107
1998-07    29.564120
1998-08    29.954622
1998-09    28.089919
1998-10    26.418671
1998-11    22.898425
1998-12    18.395948
Name: Temperature, dtype: float64
```



Year	Warmest_months	Coldest_months	Highest_monthly_mean_C	Lowest_monthly_mean_C	Valid_months	Included_in_frequency
1998	[8]	[1]	29.954622	15.233447	12	True
1999	[7]	[12]	29.653387	15.611723	12	True
2000	[7]	[2]	29.250413	14.937097	12	True
2001	[8]	[12]	29.087143	16.645134	12	True
2002	[6]	[1]	28.878421	16.112745	12	True
2003	[7]	[1]	29.846122	15.025641	12	True
2004	[8]	[1]	29.465746	14.768201	12	True
2005	[7]	[1]	29.574290	15.081757	12	True
2006	[7]	[1]	29.592598	16.389222	12	True
2007	[7]	[1]	30.132561	15.723848	12	True
2008	[9]	[2]	29.177964	12.919096	12	True
2009	[8]	[1]	29.876996	14.751973	12	True
2010	[8]	[1]	29.646909	16.664912	12	True
2011	[8]	[1]	30.162803	12.176653	12	True
2012	[8]	[1]	29.918059	13.706334	12	True
2013	[8]	[12]	28.795289	15.079650	12	True
2014	[7]	[2]	30.127428	14.790909	12	True
2015	[8]	[1]	29.634590	16.259543	12	True

```
 2016            [7]            [2]              29.864247              14.544070
12                 True
 2017            [8]            [2]              29.710555              16.820238
12                 True
 2018            [7]            [1]              29.345918              15.633468
12                 True
 2019            [7]            [1]              29.511492              17.613508
12                 True
 2020            [7]           [12]              30.330054              17.315524
12                 True
 2021            [9]            [1]              29.882939              15.753226
12                 True
 2022            [7]            [2]              30.031965              14.218006
10                False
```

Month	Warmest_count	Coldest_count
1	0	15
2	0	5
3	0	0
4	0	0
5	0	0
6	1	0
7	11	0
8	10	0
9	2	0
10	0	0
11	0	0
12	0	4



3.1 [5 points] Group the data on Storm Identifie (SID), report names (NAME) of the 10 largest hurricanes according to wind speed (WMO_WIND).

```
##PS2_3_1 group the data, 10 largest hurricanes
columns = ["SID", "SEASON", "BASIN", "NAME", "ISO_TIME", "LAT", "LON", "WMO_WIND"]
df = pd.read_csv("data/ibtracs.ALL.list.v04r00.csv", usecols=columns,
                 skiprows=[1], dtype=str, keep_default_na=False)
for column in columns:
    df[column] = df[column].str.strip()
    df[column] = df[column].replace("", np.nan)

df["NAME"] = df["NAME"].replace("NOT_NAMED", "Unnamed")
df["NAME"] = df["NAME"].fillna("Unnamed")
df["BASIN"] = df["BASIN"].fillna("Unknown")
df["ISO_TIME"] = pd.to_datetime(df["ISO_TIME"])
for column in ["SEASON", "LAT", "LON", "WMO_WIND"]:
    df[column] = pd.to_numeric(df[column], errors="coerce")
df.loc[df["WMO_WIND"] < 0, "WMO_WIND"] = np.nan

print(df.head())
print("数据点数：", len(df))
print("最后一个观测时刻：", df["ISO_TIME"].max())
storm_wind = df.groupby("SID")["WMO_WIND"].max()
storm_name = df.groupby("SID")["NAME"].first()

# 两个 Series 都用 SID 作索引，会对应到同一个风暴
storms = pd.DataFrame()
storms["NAME"] = storm_name
storms["WMO_WIND"] = storm_wind
storms = storms.dropna(subset=["WMO_WIND"])
storms = storms.reset_index()
```

```
storms = storms.sort_values(["WMO_WIND", "SID"], ascending=[False, True])

print(storms.head(10))
```

```
             SID  SEASON BASIN     NAME             ISO_TIME      LAT       LON  \
0  1842298N11080    1842    NI  Unnamed 1842-10-25 03:00:00  10.9000  80.3000
1  1842298N11080    1842    NI  Unnamed 1842-10-25 06:00:00  10.8709  79.8265
2  1842298N11080    1842    NI  Unnamed 1842-10-25 09:00:00  10.8431  79.3524
3  1842298N11080    1842    NI  Unnamed 1842-10-25 12:00:00  10.8188  78.8772
4  1842298N11080    1842    NI  Unnamed 1842-10-25 15:00:00  10.8000  78.4000

   WMO_WIND
0       NaN
1       NaN
2       NaN
3       NaN
4       NaN
数据点数： 707177
最后一个观测时刻： 2022-10-13 00:00:00
                SID      NAME  WMO_WIND
5242  2015293N13266  PATRICIA     185.0
1982  1980214N11330     ALLEN     165.0
655   1935241N23291   Unnamed     160.0
2710  1988253N12306   GILBERT     160.0
3593  1997253N12255     LINDA     160.0
4341  2005289N18282     WILMA     160.0
5588  2019236N10314    DORIAN     160.0
3690  1998295N12284     MITCH     155.0
4327  2005261N21290      RITA     155.0
4704  2009288N07267      RICK     155.0
```

3.2 [5 points] List the wind speed (WMO_WIND) of the 20 strongest-wind hurricanes.

```
##PS2_3_2 20 strongest hurricanes
top20_storms = storms.head(20)
print(top20_storms)
```

```
                SID      NAME  WMO_WIND
5242  2015293N13266  PATRICIA     185.0
1982  1980214N11330     ALLEN     165.0
655   1935241N23291   Unnamed     160.0
2710  1988253N12306   GILBERT     160.0
3593  1997253N12255     LINDA     160.0
4341  2005289N18282     WILMA     160.0
5588  2019236N10314    DORIAN     160.0
3690  1998295N12284     MITCH     155.0
4327  2005261N21290      RITA     155.0
4704  2009288N07267      RICK     155.0
```

```
5405  2017242N16333      IRMA     155.0
618   1932304N15305   Unnamed     150.0
934   1955265N13307     JANET     150.0
1027  1959249N19181     PATSY     150.0
1267  1969226N18280   CAMILLE     150.0
1717  1977242N27272     ANITA     150.0
1906  1979238N12324     DAVID     150.0
3100  1992230N11325    ANDREW     150.0
3287  1994222N11267      JOHN     150.0
4309  2005236N23285   KATRINA     150.0
```

3.3 [5 points] Count all datapoints by Basin.

```
##PS2_3_3 basin count
basin_counts = df.groupby("BASIN").size()
basin_counts = basin_counts.sort_values(ascending=False)
print(basin_counts.to_frame("Datapoints"))
print("各海盆合计：", basin_counts.sum())
```

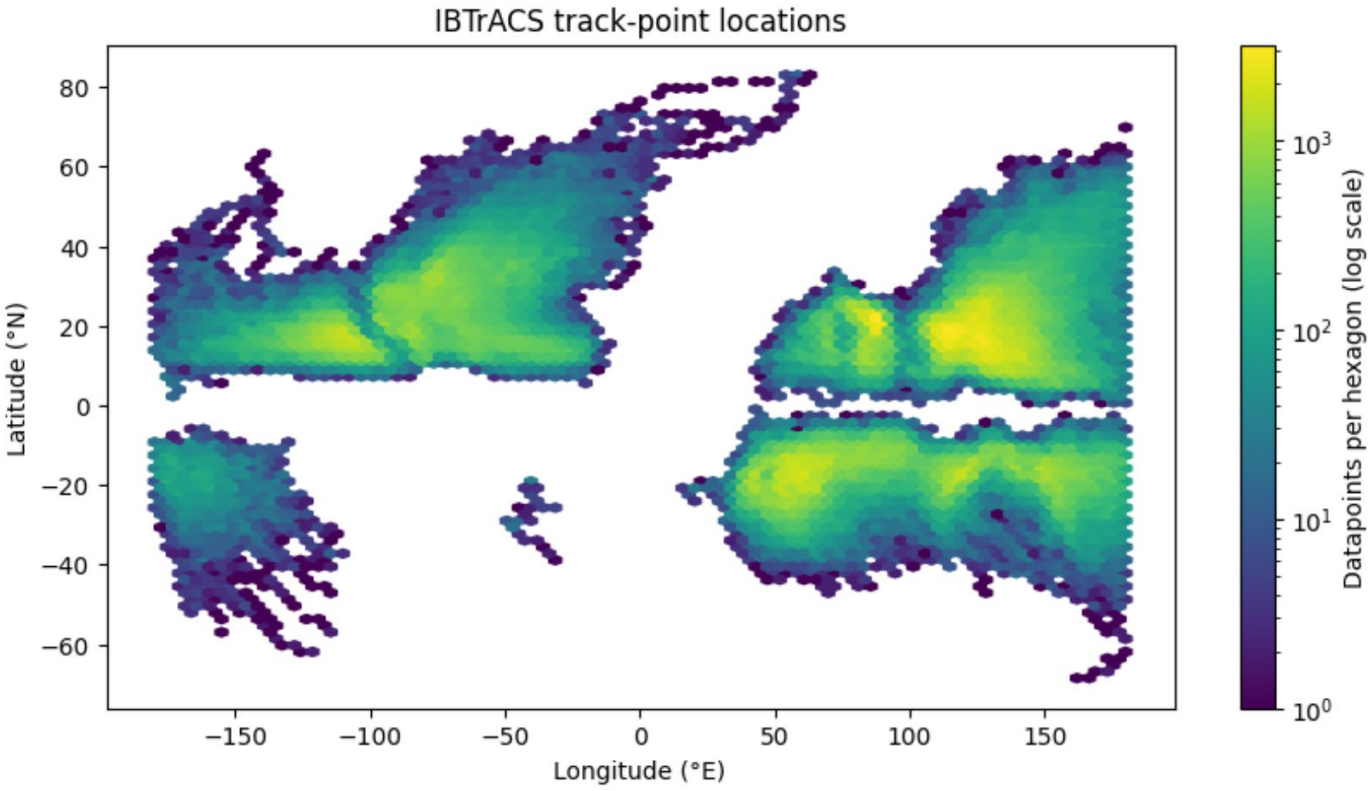
```
       Datapoints
BASIN
WP         236576
SI         160668
NA         124881
SP          67119
EP          62412
NI          55402
SA            119
各海盆合计： 707177
```

3.4 [5 points] Make a hexbin plot of the location of datapoints in Latitude and Longitude.

```
##PS2_3_4 Latitude and Longitude.
positions = df.dropna(subset=["LAT", "LON"])
positions = positions[(positions["LAT"] >= -90) & (positions["LAT"] <= 90)]
positions = positions[(positions["LON"] >= -180) & (positions["LON"] <= 180)]

plt.figure(figsize=(10, 5))
plt.hexbin(positions["LON"], positions["LAT"], gridsize=80, mincnt=1, bins="log")
plt.colorbar(label="Datapoints per hexagon (log scale)")
plt.xlabel("Longitude (°E)")
```

```
plt.ylabel("Latitude (°N)")
plt.title("IBTrACS track-point locations")
#plt.tight_layout()
plt.show()
```

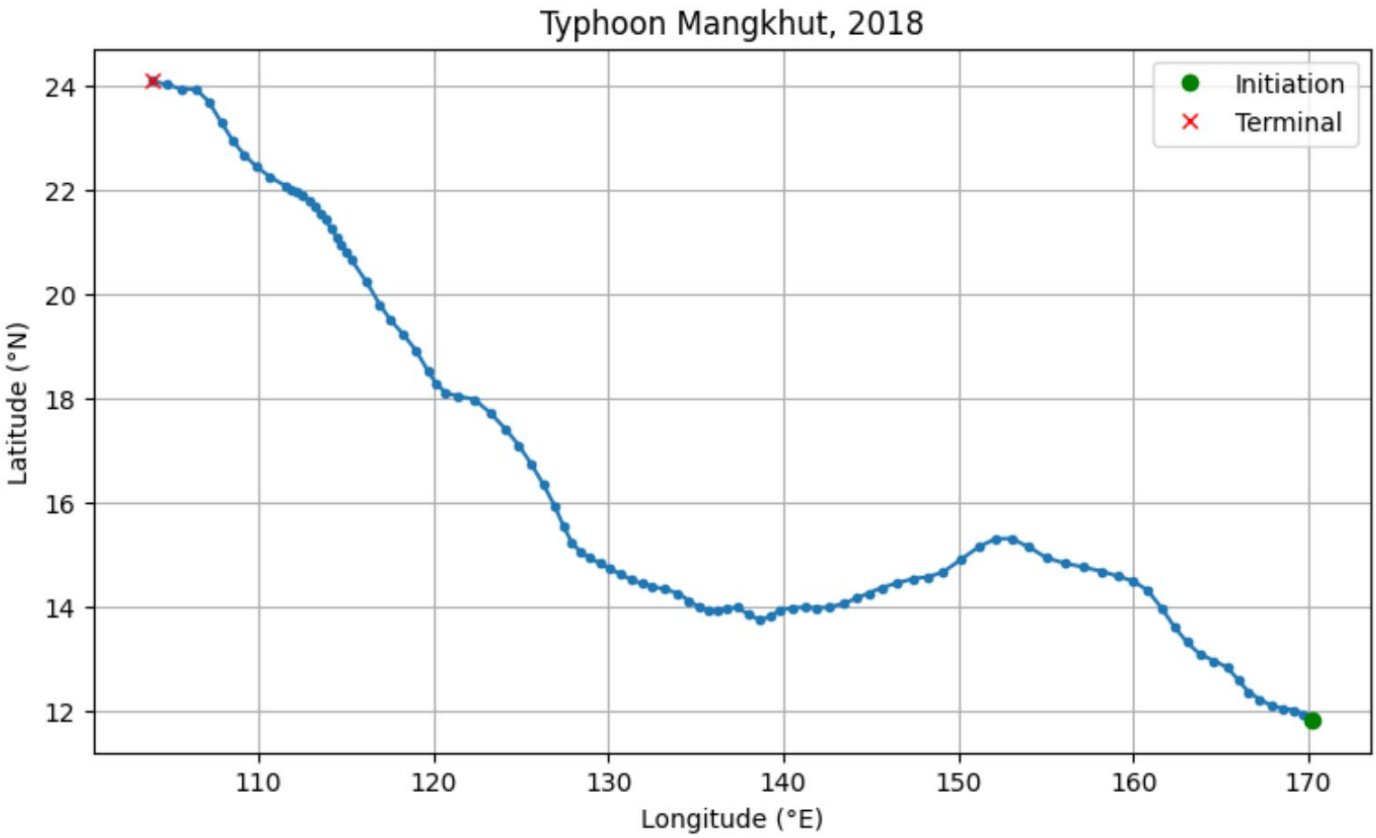


3.5 [5 points] Find Typhoon Mangkhut (from 2018) and plot its track.

```
##PS2_3_5 Typhoon Mangkhut
mangkhut = df[df["NAME"] == "MANGKHUT"]
mangkhut = mangkhut[mangkhut["SEASON"] == 2018]
mangkhut = mangkhut.sort_values("ISO_TIME")
mangkhut = mangkhut.dropna(subset=["LAT", "LON"])

plt.figure(figsize=(9, 5))
plt.plot(mangkhut["LON"], mangkhut["LAT"], "o-", markersize=3)
plt.plot(mangkhut["LON"].iloc[0], mangkhut["LAT"].iloc[0], "go", label="Initiation")
plt.plot(mangkhut["LON"].iloc[-1], mangkhut["LAT"].iloc[-1], "rx", label="Terminal")
plt.xlabel("Longitude (°E)")
plt.ylabel("Latitude (°N)")
```

```
plt.title("Typhoon Mangkhut, 2018")
plt.legend()
plt.grid(True)
#plt.tight_layout()
plt.show()
```



3.6 [5 points] Create a filtered dataframe that contains only data since 1970 from the Western North Pacific (“WP”) and Eastern North Pacific (“EP”) Basin. Use this for the rest of the problem set.

```
##PS2_3_6 WP to EP basin
pacific = df[df["ISO_TIME"] >= "1970-01-01"]
pacific = pacific[(pacific["BASIN"] == "WP") | (pacific["BASIN"] == "EP")]
pacific = pacific.copy()

print("筛选后的数据点数：", len(pacific))
print(pacific.head())
```

```
筛选后的数据点数： 176352
```

```
                 SID  SEASON BASIN   NAME            ISO_TIME      LAT  \
350394  1970050N07151    1970    WP  NANCY 1970-02-19 00:00:00  7.00000
350395  1970050N07151    1970    WP  NANCY 1970-02-19 03:00:00  7.24752
350396  1970050N07151    1970    WP  NANCY 1970-02-19 06:00:00  7.50000
350397  1970050N07151    1970    WP  NANCY 1970-02-19 09:00:00  7.75747
350398  1970050N07151    1970    WP  NANCY 1970-02-19 12:00:00  8.00000

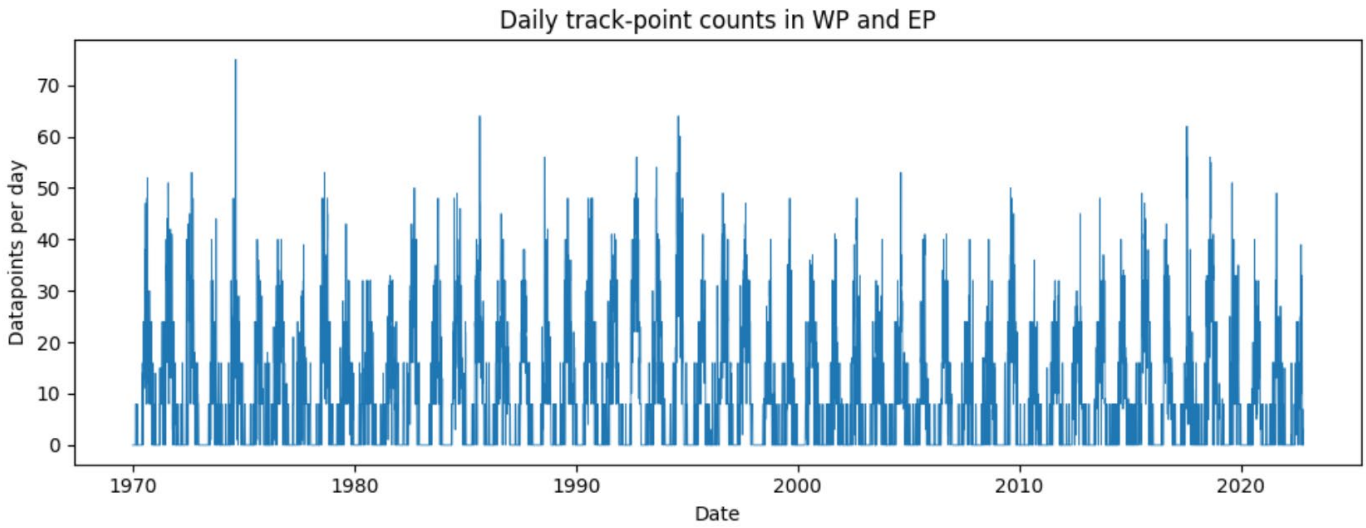
            LON  WMO_WIND
350394  151.400       NaN
350395  151.205       NaN
350396  151.000       NaN
350397  150.772       NaN
350398  150.500       NaN
```

3.7 [5 points] Plot the number of datapoints per day.

```
##PS2_3_7 the day of year
pacific_time = pacific.set_index("ISO_TIME")
daily_count = pacific_time.resample("D").size()
dates = pd.date_range("1970-01-01", "2022-10-13", freq="D")
daily_count = daily_count.reindex(dates, fill_value=0)

daily = pd.DataFrame()
daily["Count"] = daily_count

plt.figure(figsize=(10, 4))
plt.plot(daily.index, daily["Count"], linewidth=0.6)
plt.xlabel("Date")
plt.ylabel("Datapoints per day")
plt.title("Daily track-point counts in WP and EP")
plt.tight_layout()
plt.show()
print(daily.head())
```



	Count
1970-01-01	0
1970-01-02	0
1970-01-03	0
1970-01-04	0
1970-01-05	0

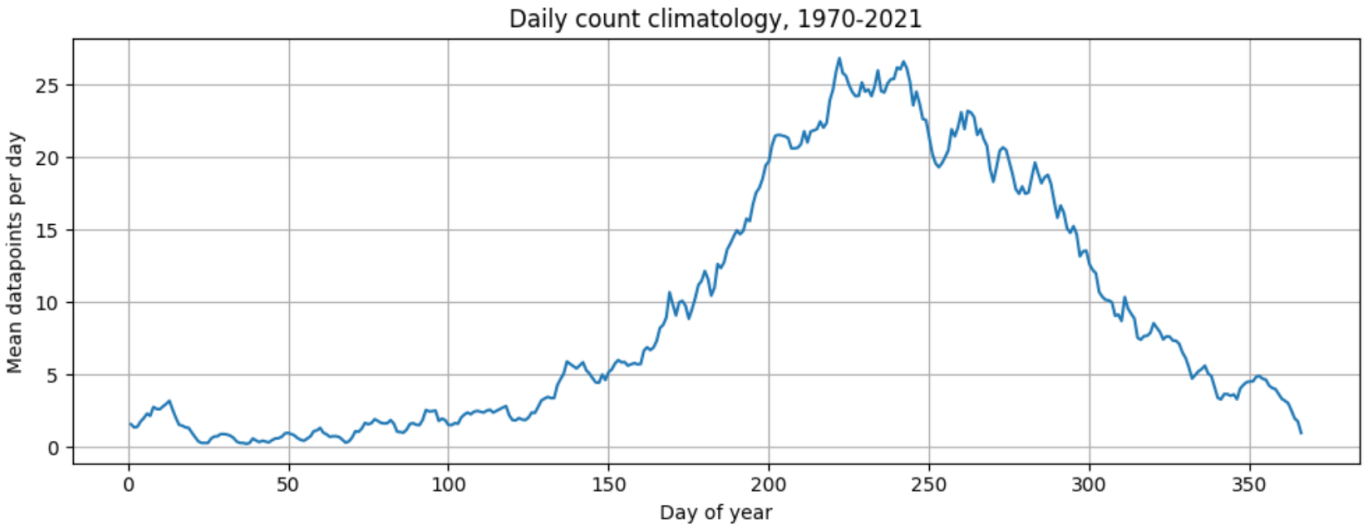
3.8 [5 points] Calculate the climatology of datapoint counts as a function of day of year. The day of year is the sequential day number starting with day 1 on January 1st.

```
##PS2_3_8 climatology
daily["Day_of_year"] = daily.index.dayofyear
baseline = daily["1970":"2021"]
climatology = baseline.groupby("Day_of_year")["Count"].mean()

print(climatology.head(10).to_frame("Mean_count"))
print("气候态计数最大的日序号：", climatology.idxmax())

plt.figure(figsize=(10, 4))
plt.plot(climatology.index, climatology.values)
plt.xlabel("Day of year")
plt.ylabel("Mean datapoints per day")
plt.title("Daily count climatology, 1970-2021")
plt.grid(True)
plt.tight_layout()
plt.show()
```

```
             Mean_count
Day_of_year
1              1.596154
2              1.384615
3              1.423077
4              1.788462
5              2.019231
6              2.326923
7              2.173077
8              2.788462
9              2.653846
10             2.634615
气候态计数最大的日序号： 222
```



3.9 [5 points] Calculate the anomaly of daily counts from the climatology.

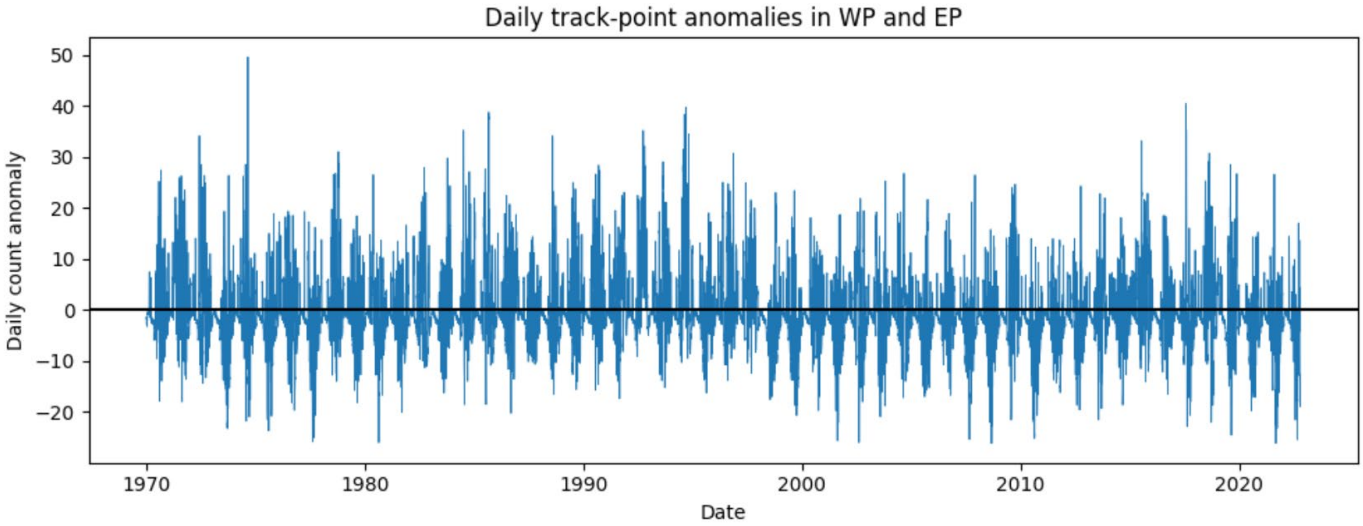
```
##PS2_3_9 anomaly of daily counts
normal_values = []
for day in daily["Day_of_year"]:
    normal = climatology.loc[day]
    normal_values.append(normal)

daily["Climatology"] = normal_values
daily["Anomaly"] = daily["Count"] - daily["Climatology"]
print(daily.head())

plt.figure(figsize=(10, 4))
plt.plot(daily.index, daily["Anomaly"], linewidth=0.6)
plt.axhline(0, color="black")
plt.xlabel("Date")
```

```
plt.ylabel("Daily count anomaly")
plt.title("Daily track-point anomalies in WP and EP")
plt.tight_layout()
plt.show()
```

	Count	Day_of_year	Climatology	Anomaly
1970-01-01	0	1	1.596154	-1.596154
1970-01-02	0	2	1.384615	-1.384615
1970-01-03	0	3	1.423077	-1.423077
1970-01-04	0	4	1.788462	-1.788462
1970-01-05	0	5	2.019231	-2.019231

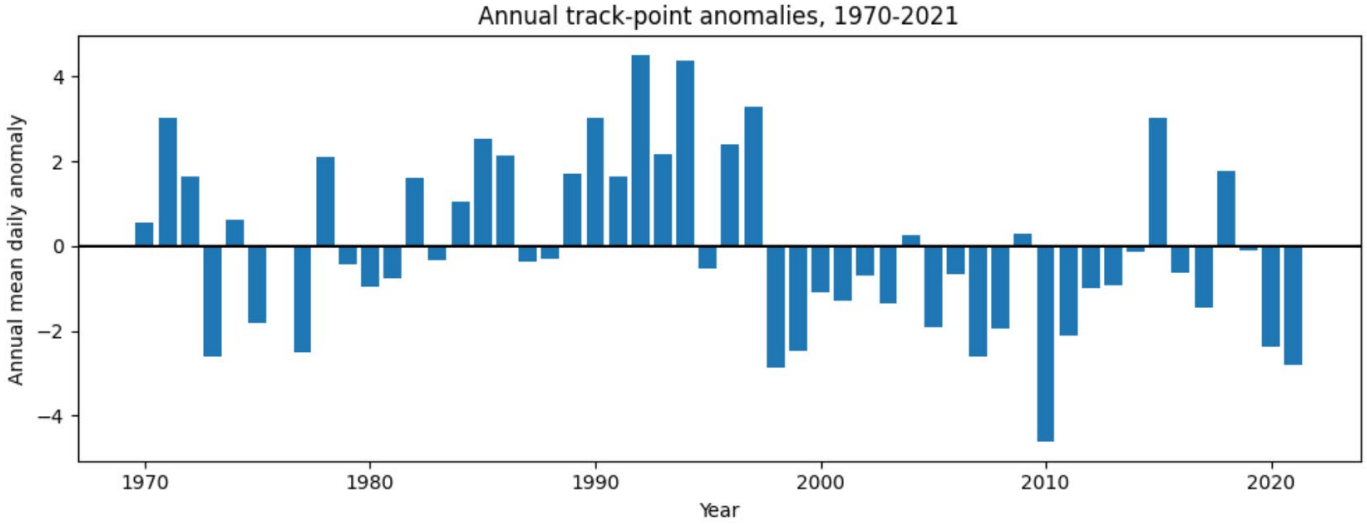


3.10 [5 points] Resample the anomaly timeseries at annual resolution and plot. So which years stand out as having anomalous hurricane activity?

```
##PS2_3_10 anomaly timeseries at annual resolution
annual_anomaly = daily["Anomaly"].resample("YS").mean()
annual_anomaly = annual_anomaly["1970":"2021"]

plt.figure(figsize=(10, 4))
plt.bar(annual_anomaly.index.year, annual_anomaly.values)
plt.axhline(0, color="black")
plt.xlabel("Year")
plt.ylabel("Annual mean daily anomaly")
plt.title("Annual track-point anomalies, 1970-2021")
plt.tight_layout()
plt.show()
```

```
high_years = annual_anomaly.sort_values(ascending=False)
low_years = annual_anomaly.sort_values()
print("距平最高的五年：")
print(high_years.head(5).to_frame())
print("距平最低的五年：")
print(low_years.head(5).to_frame())
```




```
距平最高的五年：
             Anomaly
1992-01-01  4.498686
1994-01-01  4.376765
1997-01-01  3.291834
1990-01-01  3.034299
2015-01-01  3.028820
距平最低的五年：
             Anomaly
2010-01-01 -4.609536
1998-01-01 -2.869810
2021-01-01 -2.790358
2007-01-01 -2.601317
1973-01-01 -2.595838
```

4.1 [5 points] Load the csv, XLS, or XLSX file, and clean possible data points with missing values or bad quality.

```
## PS2_4_1
## 全球温度距平数据集
global_data = pd.read_csv("data/global_temperature.csv", comment="#", dtype=str)
global_data["Date"] = pd.to_datetime(global_data["Date"], format="%Y%m",
errors="coerce")
global_data["Anomaly"] = pd.to_numeric(global_data["Departure from Average"],
errors="coerce")
global_data = global_data.dropna(subset=["Date", "Anomaly"])
global_data = global_data[global_data["Anomaly"] > -99]
global_data = global_data.drop_duplicates()
print("重复月份数：", global_data["Date"].duplicated().sum())
##数据集删除缺失、极端占位值、重复行。转换日期、数值
global_data = global_data.set_index("Date")
global_data = global_data.sort_index()
global_temp = global_data["Anomaly"]
months = pd.date_range("1880-01-01", "2025-12-01", freq="MS")
global_temp = global_temp.reindex(months)

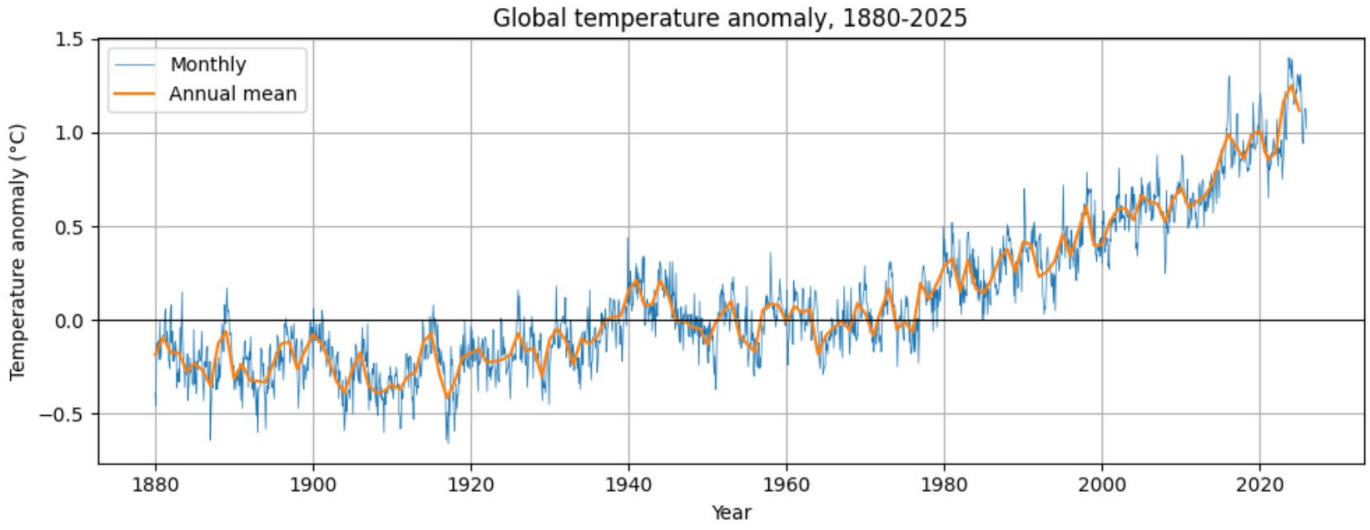
print("有效月份数：", global_temp.count())
print("缺失月份数：", global_temp.isna().sum())
print(global_temp.head().to_frame())
```

```
重复月份数： 0
有效月份数： 1752
缺失月份数： 0
            Anomaly
1880-01-01    -0.39
1880-02-01    -0.46
1880-03-01    -0.14
1880-04-01    -0.12
1880-05-01    -0.16
```

4.2 [5 points] Plot the time series of a certain variable.

```
## PS2_4_2 anomaly timeseries
global_annual = global_temp.resample("YS").mean()

plt.figure(figsize=(10, 4))
plt.plot(global_temp.index, global_temp.values, linewidth=0.5, label="Monthly")
plt.plot(global_annual.index, global_annual.values, linewidth=1.5, label="Annual mean")
plt.axhline(0, color="black", linewidth=0.8)
plt.xlabel("Year")
plt.ylabel("Temperature anomaly (°C)")
plt.title("Global temperature anomaly, 1880-2025")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```



4.3 [5 points] Conduct at least 5 simple statistical checks with the variable, and report your findings.

```
## PS2_4_3 basic analysis of 7 indexes
number = global_temp.count()
```

```
average = global_temp.mean()
median = global_temp.median()
std = global_temp.std()
minimum = global_temp.min()
maximum = global_temp.max()
value_range = maximum - minimum

statistics = pd.DataFrame()
statistics["Statistic"] = ["Count", "Mean", "Median", "Std", "Minimum", "Maximum",
"Range"]
statistics["Value"] = [number, average, median, std, minimum, maximum, value_range]
print(statistics)
print("最小值出现时间：", global_temp.idxmin())
print("最大值出现时间：", global_temp.idxmax())

early_mean = global_temp["1880":"1909"].mean()
recent_mean = global_temp["1996":"2025"].mean()
print("1880-1909 年平均距平：", round(early_mean, 3), "°C")
print("1996-2025 年平均距平：", round(recent_mean, 3), "°C")
print("后期减去前期：", round(recent_mean - early_mean, 3), "°C")
```

```
  Statistic        Value
0     Count  1752.000000
1      Mean     0.103687
2    Median     0.010000
3       Std     0.392286
4   Minimum    -0.660000
5   Maximum     1.400000
6     Range     2.060000
最小值出现时间： 1917-03-01 00:00:00
最大值出现时间： 2023-09-01 00:00:00
1880-1909 年平均距平： -0.235 °C
1996-2025 年平均距平： 0.727 °C
后期减去前期： 0.961 °C
```